NutriPlanner API Technical Report

1. Project Overview

This project implements a database-driven Web API: the NutriPlanner API. Designed for ingredient management, recipe curation, nutritional analysis, and recommendation decision-making, the system extends beyond foundational CRUD operations to provide interpretable nutritional and cost computations, complemented by a visualization module for integrated public data insights. The primary objective is to deliver a verifiable, interpretable, and demonstrable backend system tailored for academic coursework.

Aligned with course objectives, the project satisfies core requirements for database integration, Web API design, and comprehensive documentation, while extending capabilities in analytical depth and visual presentation.

2. Architecture and Technology Stack

2.1 Architectural Design

The system adopts a modular, layered architecture comprising auth, ingredients, recipes, and analytics modules. Each module adheres to a strict separation of concerns: the routing layer handles protocol and parameter validation, the service layer executes business rules and computations, and the data access layer manages persistence and transaction boundaries. Cross-cutting concerns, including authentication, input validation, error handling, and security middleware, are implemented globally.

This design yields three key benefits:

First, maintainability. Feature extensions are isolated within module boundaries, preventing regression. Second, interpretability. Request lifecycles can be clearly traced and explained during oral assessments. Third, testability. Core business logic is centralized in the service layer, facilitating unit and integration testing.

2.2 Technology Stack and Rationale

- Node.js + TypeScript + Express: High iteration velocity and mature ecosystem; TypeScript's static type system mitigates runtime errors.
- sql.js (SQLite): Meets SQL coursework requirements with minimal dependencies, ensuring reproducible local execution.
- Zod: Centralized request validation to prevent malformed data from reaching core logic.
- jsonwebtoken + bcryptjs: Secure authentication flow and cryptographic password hashing.
- helmet / express-rate-limit / cors: Implementation of foundational security headers and traffic control.
- Vitest + Supertest: Comprehensive test coverage for critical API endpoints, supporting regression validation.
- Plotly: Enables complex 3D and statistical visualizations for the insights module.

The project employs a local-first demonstration strategy, eliminating external deployment dependencies while fully showcasing system capabilities.

3. Database Design and Rationale

3.1 Data Model

The core relational schema includes:

- users: User credentials, roles, and password hashes.
- refresh_tokens: Session management, expiration tracking, and revocation states.
- ingredients: Nutritional parameters, cost metrics, and allergen tags.
- recipes: Primary recipe metadata.
- recipe_ingredients: Junction table mapping many-to-many relationships between recipes and ingredients, including precise gram measurements.

The recipe_ingredients table is a critical design choice. Nutritional values and costs are dynamically aggregated based on ingredient quantities rather than stored as static fields. This enables the API to perform accurate, real-time composite calculations.

3.2 Relational Modeling Value

The adoption of a relational SQL database is justified by:

- Referential integrity and traceability via foreign key constraints.
- Efficient JOIN operations to extract contextual relationships between recipes, ingredients, and nutritional metrics.
- A structured schema that aligns with academic grading criteria for design justification.
- Transparent query and computation pathways, facilitating result interpretation and defense.

3.3 Data Initialization Strategy

Data provisioning follows a dual-track approach:

- Business Demonstration Data: Automated seed scripts initialize administrative accounts and foundational ingredient datasets upon system startup.
- Visualization Public Data: Custom scripts aggregate and normalize open-source datasets into unified JSON structures for the insights module.

This strategy ensures both functional testability of the API and compliance with data integration and visualization requirements.

4. Core Features and Algorithmic Implementation

4.1 CRUD and Access Control

The system implements full CRUD operations for ingredients and recipes, enforcing a "public read, authenticated write" policy. High-risk operations (e.g., ingredient deletion) are restricted by role-based access control (RBAC). The authentication flow supports registration, login, token refresh, and session termination, meeting documentation and demonstration standards.

4.2 Nutritional Analysis Algorithm

Nutritional aggregation is computed dynamically based on ingredient mass:

$$N_total = \Sigma_i(grams_i/100 \times N_i, 100g)$$

Leveraging the 4-4-9 caloric conversion rule, the system calculates macronutrient energy distribution, deriving:

- Per-serving caloric and macronutrient totals.

- macroBalance: A metric quantifying macronutrient equilibrium.
- proteinDensity: Protein mass per calorie unit.

This computational pipeline maps recipe composition directly to interpretable health metrics, eliminating reliance on subjective tagging.

4.3 Recommendation Scoring Model

The recommendation endpoint employs a two-stage pipeline: hard constraint filtering followed by weighted scoring and ranking.

Hard Constraints:
- minProteinPerServing
- maxCostPerServing
- excludeAllergens

Scoring Formula:

$$\text{Score} = 0.35 \times \text{macroBalance} + 1.6 \times \text{proteinDensity} - 8 \times \text{costPerServing} - 35 \times \text{caloriePenalty}$$

The caloriePenalty quantifies deviation from target caloric intake. This model balances health objectives and economic efficiency, with each component independently interpretable for academic reporting.

4.4 Leaderboards and Cost Tiering

The system supports dual leaderboards: protein density ranking and cost-efficiency ranking (protein-to-cost ratio). Cost analysis results are discretized into categorical tiers (budget, moderate, premium), mapping continuous values to readable classifications to enhance business interpretability.

5. Testing Approach

Testing was performed with both automated integration tests and end-to-end manual verification. Automated tests (Vitest + Supertest) cover:
- health and root availability,
- authentication flow (register/login/me/refresh/logout),
- protected-route access control (authenticated vs unauthenticated),
- ingredients CRUD and validation behavior,
- recipes CRUD and relational integrity checks,
- analytics endpoints (nutrition, cost, leaderboard, recommendations),
- standard error responses (400/401/403/404/409).

Engineering safety checks were executed before submission:
- npm run typecheck
- npm run build
- npm test

In addition, a one-command acceptance script (scripts/manual-check.ps1) validates the full runtime workflow and prints PASS/FAIL summaries, ensuring the system remains stable after

iterative changes.

6. Insights Visualization Module

The /insights module visualizes integrated public data to demonstrate data processing and analytical outcomes. It features 3D scatter plots, radar charts, leaderboards, and correlation heatmaps, with interactive controls for region filtering, risk thresholds, and dynamic Top-N selection.

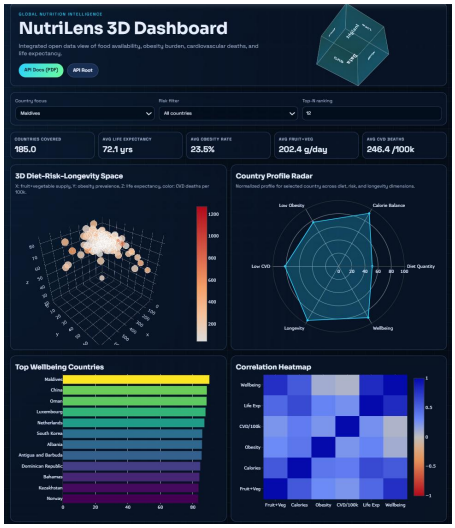

Figure 1 insights

7. Challenges, Improvements, and Limitations

7.1 Key Challenges

1. Transaction Boundary Management: Ensuring precise commit/rollback timing during compound write operations.
2. API-First Paradigm Adoption: Bridging the gap for users accustomed to UI-based authentication through comprehensive documentation.
3. Explainability vs. Personalization: Rule-based models offer high transparency but limited adaptive depth.

7.2 Current Limitations

- Recommendation weights remain manually configured.
- Absence of historical user behavior tracking.
- Local-first architecture lacks production-grade monitoring pipelines.
- Recommendation quality is constrained by base dataset richness.

7.3 Future Work

- Incorporate user preference feedback loops for iterative optimization.
- Establish an experimental framework for data-driven hyperparameter tuning.
- Implement CI/CD pipelines and code coverage thresholds.
- Expand public data ingestion to enhance semantic understanding and personalization.

8. GenAI Usage Declaration

8.1 Tools Utilized

- ChatGPT / Codex (GPT series models)

8.2 Validation and Implementation Workflow

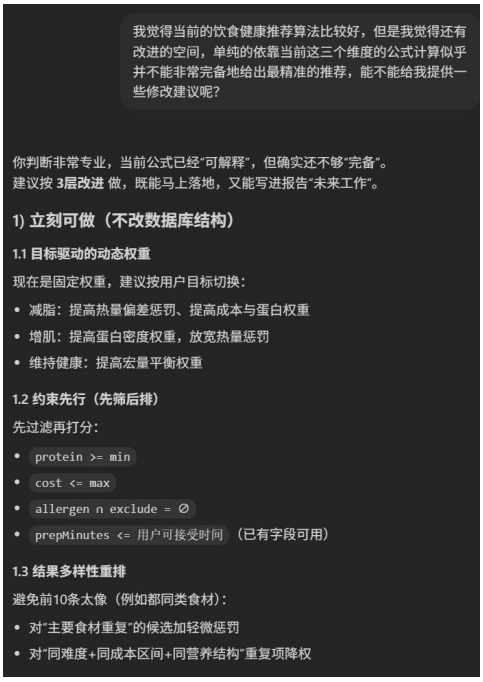

Figure 2 Optimaisation

Figure 3 Check

In the area of recommendation algorithm optimisation, AI provided valuable suggestions on dynamic weight adjustment and constraint-based filtering strategies. Based on different user goals such as fat loss, muscle gain, and health maintenance, AI recommended implementing target-driven dynamic weights rather than fixed weights. Additionally, AI suggested applying diversity penalties to avoid repetitive recommendations in the top results, which significantly improved the practical usability of the recommendation system.

Regarding engineering quality and testing, AI assisted in diagnosing and resolving TypeScript errors, particularly type safety issues in error handling. When encountering the rollbackError type checking problem, AI guided me to implement type narrowing before accessing properties. AI also helped structure the comprehensive testing workflow, including typecheck, build, and test commands, ensuring that all modifications maintained code quality and system stability throughout the development process.

9. Conclusion

This project successfully fulfills the core objectives of the coursework: a database-driven API, verifiable CRUD operations, robust authentication and error handling, and comprehensive documentation. Beyond baseline requirements, the system delivers interpretable nutritional analysis, a transparent recommendation scoring model, and an insights visualization module that strengthens data integration and presentation. From an engineering perspective, the architecture demonstrates clear modular boundaries, reproducible testing protocols, and maintainable submission standards.